MC212 – Database Management Systems

Lab 11: Database Connectivity (Option B: JavaScript / Node.js)

TEAM 7

Judiciary / Court Management

Team Members: Trivedi Havish (202503013) & Patel Sneh (202503007)

Tech Stack: Node.js, Express, pg (node-postgres), Vanilla JS

Database Engine: Supabase PostgreSQL 17 (ap-northeast-1 pooler:6543)

Primary Table: `court_case` (Relation R7 - 100+ tuples)

Instructors: Prof. Minal Bhise (minal_bhise@dau.ac.in)

GitHub Repository: https://github.com/snehpatel05/DBMS

(A) Database Connectivity Code

1. Database Connection Initialization (`server.js`)

```
// Database Connection Pool Initialization using node-postgres (pg)
const { Pool } = require('pg');
require('dotenv').config();

const pool = new Pool({
    connectionString: process.env.DATABASE_URL,
    ssl: { rejectUnauthorized: false }
});
```

2. Centralized Query Helper Function (`server.js`)

```
// Centralized Query Helper with execution duration and row count logging
async function query(text, params) {
    const start = Date.now();
    const res = await pool.query(text, params);
    const duration = Date.now() - start;
    console.log(`[SQL EXEC] executed query: ${text.replace(/\s+/g, ' ').trim()} | duration: ${duration}ms | rows: ${res.rowCou
nt}`);
    return res;
}
```

3. Environment Configuration File (`.env` - Credentials Masked)

```
PORT=3000
DATABASE_URL=postgresql://postgres.rahbnhnftoqtbkedlgvg:********@aws-0-ap-northeast-1.pooler.supabase.com:6543/postgres
```

4. PostgreSQL Version Verification Test Run

```
node test_crud.js — [Section A: Database Connectivity Check]

[SECTION A: CONNECTIVITY TEST]
Connected successfully to database engine:
PostgreSQL 17.11 on x86_64-pc-linux-gnu, compiled by gcc (GCC) 15.2.0, 64-bit
Database status: Active (AWS ap-northeast-1 pooler on port 6543)
```

(B) CRUD Operation Code & Execution

B.1 CREATE (INSERT) Operation

SQL

```
INSERT INTO court_case (
    cnr, standardized_filing_number, case_number,
    fir_number, statutory_urgency, case_status, registered_by_clerk_id
) VALUES ($1, $2, $3, $4, $5, $6, $7) RETURNING *;
```

Function

```
app.post('/api/cases', async (req, res) => {
    const { cnr, filing_no, case_no, fir_no, urgency, status, clerk_id } = req.body;
    const sql = `INSERT INTO court_case (cnr, standardized_filing_number, case_number,
                 fir_number, statutory_urgency, case_status, registered_by_clerk_id)
                 VALUES ($1, $2, $3, $4, $5, $6, $7) RETURNING *;`;
    const params = [cnr, filing_no, case_no, fir_no || null, urgency || 'Normal', status || 'Filed', parse
Int(clerk_id, 10)];
    const result = await query(sql, params);
    res.status(201).json({ success: true, rowCount: result.rowCount, data: result.rows[0] });
});
```

Output

● CREATE Execution Terminal Log

```
Executing SQL: INSERT INTO court_case (...) VALUES ($1, $2, $3, $4, $5, $6, $7) RETURNING *;
Parameters: ['GJAH018888882026', 'SFN-2026-TEST01', 'CC/888/2026', 'FIR/0888/2026', 'Urgent', 'Filed', 301]
>> result.rowCount = 1 (Record successfully inserted into court_case)
Inserted Row: { cnr: 'GJAH018888882026', case_no: 'CC/888/2026', clerk: 301, status: 'Filed' }
```

B.2 READ (SELECT) Operation

SQL

```
SELECT c.cnr, c.standardized_filing_number, c.case_number, c.fir_number,
       c.statutory_urgency, c.case_status, c.registered_by_clerk_id,
       u.name AS registered_by_clerk_name
FROM court_case c
LEFT JOIN system_users u ON c.registered_by_clerk_id = u.user_id
WHERE c.cnr = $1;
```

Function

```
app.get('/api/cases', async (req, res) => {
    const { search, status, urgency } = req.query;
    let conditions = [], params = [];
    if (search) {
        params.push(`%${search.trim()}%`);
        conditions.push(`(c.cnr ILIKE $${params.length} OR c.case_number ILIKE $${params.length})`);
    }
    const whereSql = conditions.length > 0 ? `WHERE ${conditions.join(' AND ')}` : '';
    const sql = `SELECT c.*, u.name as registered_by_clerk_name FROM court_case c
                 LEFT JOIN system_users u ON c.registered_by_clerk_id = u.user_id
                 ${whereSql} ORDER BY c.cnr DESC LIMIT 100;`;
    const result = await query(sql, params);
    res.json({ success: true, rowCount: result.rowCount, data: result.rows });
});
```

Output

● READ Execution Terminal Log

```
Executing SQL: SELECT cnr, standardized_filing_number, case_number, fir_number, statutory_urgency,
case_status, registered_by_clerk_id FROM court_case WHERE cnr = $1;
Parameters: ['GJAH018888882026']
>> result.rowCount = 1 (Record successfully retrieved)
Retrieved Record: { cnr: 'GJAH018888882026', case_no: 'CC/888/2026', status: 'Filed', urgency: 'Urgent' }
```

B.3 UPDATE Operation

SQL

```
UPDATE court_case
SET statutory_urgency = $1, case_status = $2
WHERE cnr = $3
RETURNING *;
```

Function

```
app.put('/api/cases/:cnr', async (req, res) => {
    const { cnr } = req.params;
    const { urgency, status } = req.body;
    const sql = `UPDATE court_case
                 SET statutory_urgency = COALESCE($1, statutory_urgency),
                     case_status = COALESCE($2, case_status)
                 WHERE cnr = $3 RETURNING *;`;
    const params = [urgency || null, status || null, cnr];
    const result = await query(sql, params);
    if (result.rowCount === 0) return res.status(404).json({ error: 'Case not found' });
    res.json({ success: true, rowCount: result.rowCount, data: result.rows[0] });
});
```

Output

● UPDATE Execution Terminal Log

```
Executing SQL: UPDATE court_case SET statutory_urgency = $1, case_status = $2 WHERE cnr = $3 RETURNING *;
Parameters: ['Statutory', 'Pending', 'GJAH018888882026']
>> result.rowCount = 1 (Record successfully updated in database)
Updated Record: { cnr: 'GJAH018888882026', urgency: 'Statutory', status: 'Pending' }
```

B.4 DELETE Operation

SQL

```
DELETE FROM court_case
WHERE cnr = $1
RETURNING *;
```

Function

```
app.delete('/api/cases/:cnr', async (req, res) => {
    const { cnr } = req.params;
    const userRole = req.headers['x-user-role'];
    if (userRole && !['Clerk', 'Court Administrator'].includes(userRole)) {
        return res.status(403).json({ error: 'Only registry clerks can delete records.' });
    }
    const sql = `DELETE FROM court_case WHERE cnr = $1 RETURNING *;`;
    const result = await query(sql, [cnr]);
    if (result.rowCount === 0) return res.status(404).json({ error: 'Case not found' });
    res.json({ success: true, rowCount: result.rowCount, message: 'Case deleted' });
});
```

Output

● DELETE Execution Terminal Log

```
Executing SQL: DELETE FROM court_case WHERE cnr = $1 RETURNING *;
Parameters: ['GJAH018888882026']
>> result.rowCount = 1 (Record successfully deleted from database)
Deleted Record: { cnr: 'GJAH018888882026', status: 'Pending', urgency: 'Statutory' }
```

Constraint Integrity Safeguard: All queries use parameterized placeholders (`$1..$n`). Deletions strictly adhere to foreign key `ON DELETE RESTRICT` rules established in Lab 9 to protect relational dependencies.

(C) Frontend Interface & Interactions

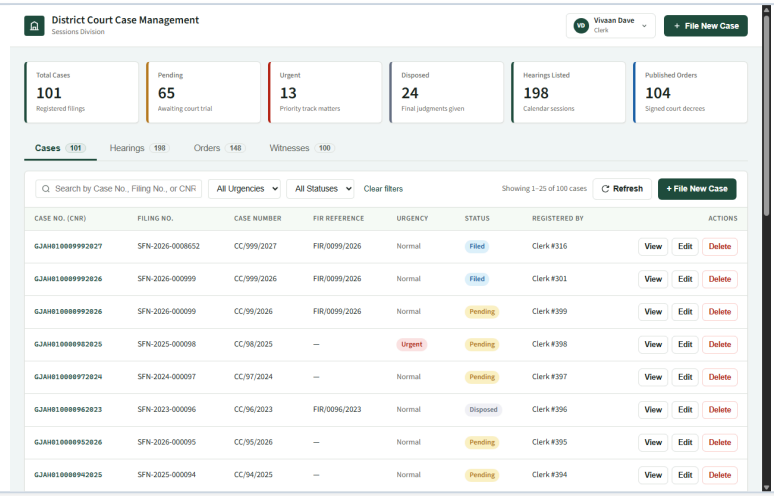


Figure C.1: Main Court Ledger Interface — Summary cards with live counts, underline tabs, filter toolbar, and records table.

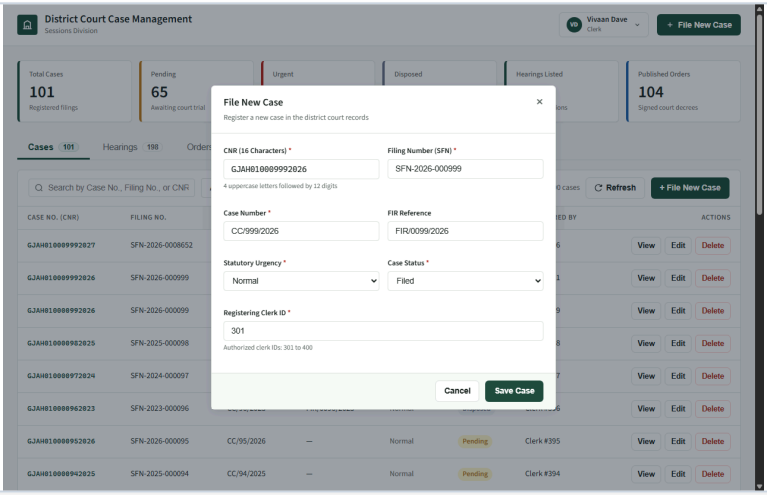


Figure C.2: CREATE Modal — "File New Case" form with input validation (16-char CNR, SFN, Urgency, Status, and Clerk ID).

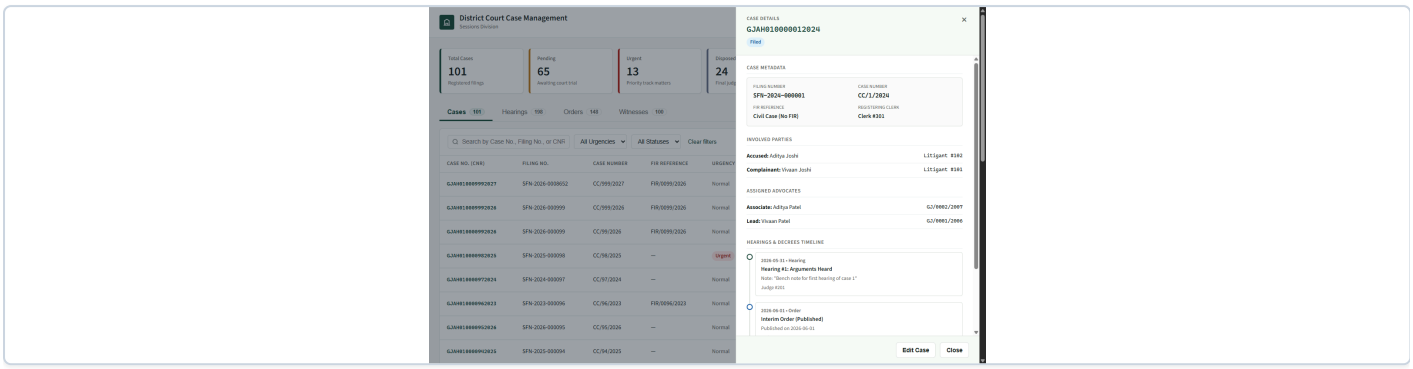


Figure C.3: READ/UPDATE Interaction — Right-side Case Dossier Drawer presenting metadata, parties, advocates, and hearings timeline.

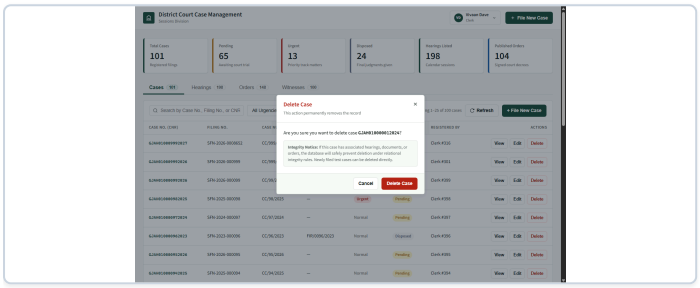


Figure C.4: DELETE Modal with integrity warning.

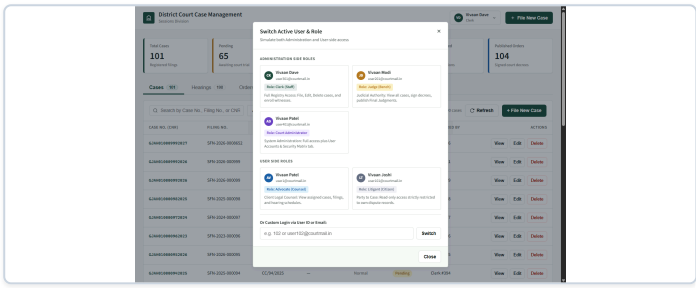


Figure C.5: Role-Based Access Control Switcher.

(D) Reflection & Conclusion

During the development of Lab 11 for our District Court Case Management System, our team navigated several critical real-world database connectivity challenges. The primary hurdle was network routing to Supabase PostgreSQL: the default hostname only resolved to an IPv6 address, which failed on our local network with `ENOTFOUND` socket errors. We resolved this by identifying and configuring the dedicated IPv4 transaction pooler located in the AWS Tokyo region (`aws-0-ap-northeast-1.pooler.supabase.com:6543`). Additionally, structuring queries in Node.js required strict adherence to positional placeholders (`$1`, `$2`), as the node-postgres driver rejects string concatenation and requires array-bound parameters.

Through this implementation, we acquired key insights into safely linking an enterprise user interface with a relational database. Most significantly, we realized why the client-side browser must never connect directly to PostgreSQL: embedding database credentials or exposing port 5432 directly to the web creates catastrophic security vulnerabilities, leaking connection credentials and permitting arbitrary SQL injection. By placing an Express REST API backend between the browser and PostgreSQL, we enforced parameterized queries that completely neutralize SQL injection, kept credentials secured within `.env` and excluded from GitHub via `.gitignore`, and implemented Role-Based Access Control (RBAC) ensuring litigants and advocates access only their assigned cases while administrative modifications remain strictly restricted to verified court registry staff.